

Test driven development and Logging framework

1. Introduction to Test Driven Development (TDD)

It is software development method in which you write automation test before the development starts.

TDD simply means a method of coding in which you **first write a test**, and it fails, then write the code to **pass the test of development**, and clean up the code.

Normal way : write code → test code

TDD : write tests → write code → improve code

Why TDD?

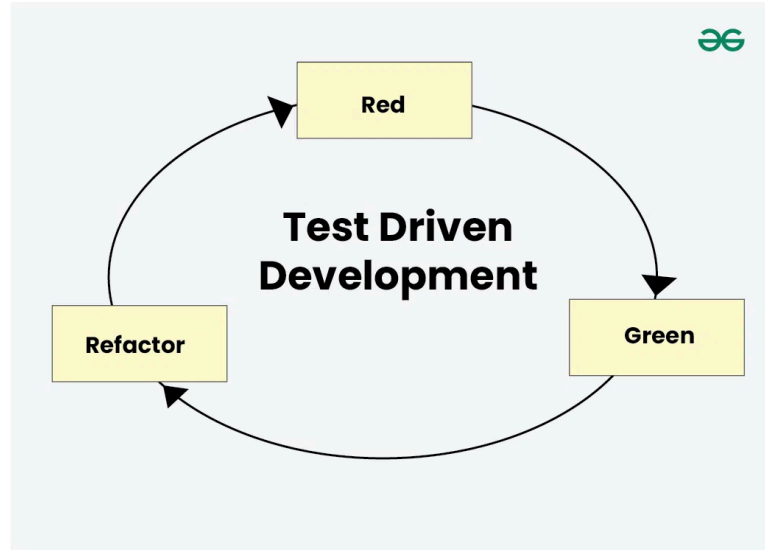
1. Finds bugs early
2. shorter dev life cycles
3. Ensure application met requirements
4. Quality of design increases

Traditional Development vs TDD

Traditional Development	Test Driven Development
Write code first	Write test first
Testing at the end	Testing throughout development
Bugs found later	Bugs found immediately
Lower code coverage	Higher code coverage
Harder maintenance	Easier maintenance

2. Process of Test Driven Development (TDD)

You write automated unit tests *before* writing the actual production code.



Run all the test cases and make sure that the new test case fails.

- **Red** - Create a test case and make it fail, Run the test cases
- **Green** - Make the test case pass by any means.
- **Refactor** - Change the code to remove duplicate/redundancy and Refactor code - This is done to remove duplication of code.

3. Approaches of Test Driven Development (TDD)

1. Inside-Out (**Detroit School of TDD** or **Classicist**)

Development starts by testing **smallest unit of code** such as functions or classes. After these units are tested they are **combined together** to build larger components until **application developed**.

How it Works

1. Write a unit test for a small method.
2. Implement the method.
3. Make the test pass.
4. Refactor the code if needed.
5. Repeat the process for other methods.
6. Integrate all the tested units to build the complete application.

Advantages

- Simple to learn and apply.
- High confidence in individual components.
- Less dependency on mocking frameworks.
- Encourages clean and reusable code.

Disadvantages

- Business requirements may not be verified until later.
- Integration issues may appear after combining components.

2. Outside-In (**London School of TDD or Mockist**)

It will focus on testing user behavior and interactions.

- Testing starts at the outermost level, such as the user interface, and works inward to the details.
- Harder to learn but ensure the code meets overall business needs.

Advantages

- Business requirements are **validated** from the **beginning**.
- Ideal for **large applications**.
- Produces well structured and loosely coupled applications.
- Encourages dependency injection.

Disadvantages

- Required good understanding
- More complex than inside-out approach
- More effort

Inside Out vs Outside In

Feature	Inside Out	Outside In
Also Known As	Detroit School / Classicist	London School / Mockist
Starting Point	Small methods and classes	User interface or business feature
Focus	Individual units	User behavior and interactions
Development Direction	Inside → Outside	Outside → Inside
Mock Usage	Minimal	Extensive
Design	Evolves during refactoring	Planned from the beginning
Difficulty	Easier	More difficult
Best For	Small and medium-sized projects	Large enterprise applications

4. Unit Testing

It is a testing **smallest unit of code**, usually one method.

ex: in calculator, testing add method, etc.. tested independently

Importance

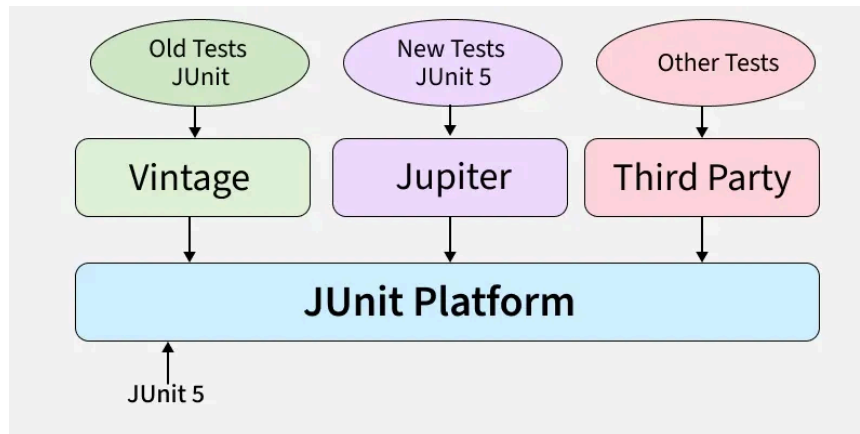
- Detect bugs earlier
- Improves code quality
- Makes debugging easier
- Prevent future errors

Unit Testing vs Integration Testing

Unit Testing	Integration Testing
Tests one class	Tests multiple classes together
Uses mocks	Uses real components
Fast	Slower
Isolates code	Tests interactions

5. JUnit Framework

JUnit is the most popular Java testing framework.



JUnit 5 is composed of three main modules:

1. JUnit Platform
2. JUnit Jupiter
3. JUnit Vintage

- **JUnit Platform**

1. **Definition:** The foundation of JUnit 5 that launches and executes test cases on the Java Virtual Machine (JVM).
2. **Purpose:** Provides a common framework for running tests written using different testing frameworks.
3. **Example:** It allows IDEs like IntelliJ IDEA and Eclipse, or build tools like Maven and Gradle, to discover and run JUnit tests.

- **JUnit Vintage**

1. Allows you to run legacy test written in JUnit 3 or 4 on the JUnit 5 Platform.

- **JUnit Jupiter**

1. It is a module that provides new programming models and feature for writing test cases in JUnit 5
2. It includes annotations, assertions, assumptions and more.

Annotations

Provide instructions to the Spring framework at runtime or compile time.

- Reduces XML-based configuration
- Improves code readability and maintainability
- Enables auto-configuration
- Simplifies dependency injection

Types in testing annotations.

- **@Test**: Marks a method as a test method.
- **@BeforeEach**: Indicates that the annotated method should be executed before each test.
- **@AfterEach**: Indicates that the annotated method should be executed after each test.
- **@BeforeAll**: Indicates that the annotated method should be executed before all tests in the test class.
- **@AfterAll**: Indicates that the annotated method should be executed after all tests in the test class.
- **@DisplayName**: Provides a custom name for the test class or test method.
- **@Disabled**: Disables the test method or class.

Assertions

JUnit 5 provides **different methods** in Assertion for checking **expected results**.

- **assertEquals(expected, actual)**: Checks if two values are equal.
- **assertTrue(condition)**: Checks if a condition is true.
- **assertFalse(condition)**: Checks if a condition is false.
- **assertNotNull(object)**: Checks if an object is not null.

6. Test Structure

- **AAA (Arrange, Act, Assert) Pattern**

- To make sure the test is readable
- Makes testing easier by separating **testing objectives, actions, results of the test.**

Why?

- Clarifies objective of each tests.
- Organizes tests for better readability
- Simplifies debugging

How to implement it?

- **Arrange** : Objective is to set to date, objects
- **Act** : Execute unit under test
- **Assert** : Verify the expected results

Arrange

Create required objects.

```
Calculatorcalculator=newCalculator();
```

Act

Call the method.

```
intresult=calculator.add(2,3);
```

Assert

Check output.

```
assertEquals(5,result);
```

Complete example

```
@Test
void testAddition(){

    // Arrange
    Calculator calculator=new Calculator();

    // Act
    int result=calculator.add(2,3);

    // Assert
    assertEquals(5,result);

}
```

Text Fixtures

Common objects shared by multiple tests.

Instead of

```
@Test
void test1(){
    Calculator c=new Calculator();
}
```

Use

```
Calculator c;

@BeforeEach
void setup(){
    c=new Calculator();
}
```

Setup and Teardown

BeforeEach

Runs before every test.


```
@BeforeEach
voidsetup(){

}
```

AfterEach

Runs after every test.

```
@AfterEach
voidcleanup(){

}
```

BeforeAll

Runs once before all tests.

```
@BeforeAll
staticvoidstart(){

}
```

AfterAll

Runs once after all tests.

```
@AfterAll
staticvoidend(){

}
```

7. Advanced JUnit Features

1. Parameterized tests

JUnit allows a single test methods to run multiple times with different inputs.

- Reuse one test method to multiple inputs.
- Code reusability

```
@ParameterizedTest
@ValueSource(strings={"555 555 55 55", "5555555555", "+1555555555"})
void testProcessValidPhones(String phone){
    assertTrue(phoneValidationService.validatePhone(phone));
}
```

2. Test suites

It is way to combine multiple test classes and run them together as single unit.

- **@Suite** annotation is used to define the class as test suite.
- Also can run specific test classes in the suite using @SelectClasses, @SelectPackages

source: <https://www.geeksforgeeks.org/software-testing/junit-5-test-suites-with-example/>

3. Test Execution

Test execution order is not in the order by default. Every Test is randomly executed one after the other.

There is 3 ways to control the order of execution

- Annotation order
- Alphanumeric order
- Random order

@Test

@Order(1)

void login(){}

Why is it useful?

Suppose you're testing an online shopping application.

1. Login
2. Add product
3. Make payment
4. Logout

It makes sense to execute them in that order.

4. Exception Testing

You want to verify the method throws expected exception on invalid inputs.

```
public int divide(int a, int b) {  
    return a / b;  
}
```

```
@Test  
void testDivideByZero() {
```

```
    assertThrows(  
        ArithmeticException.class,  
        () -> calculator.divide(5, 0)  
    );
```

```
}
```

5. Timeout testing

Sometimes a method should finish within a specified time.

```
assertTimeout(  
    Duration.ofSeconds(2),  
    () -> {  
        // code  
    }  
);
```

Why use timeout testing?

It helps ensure that performance-sensitive operations complete within an acceptable time.

Examples include:

- Database queries
- File processing
- API calls
- Search algorithms
- Sorting large datasets

8. Mockito basics

Mockito creates fake (mock) objects to isolate the class under test.

Instead of calling a real database or API, a mock object returns predefined values.

Why Mockito?

Without Mockito

```
Service
  ↓
Database
```

Testing requires a real database.

With Mockito

```
Service
  ↓
Mock Database
```

No actual database is needed.

Stubbing

Specify what the mock should return.

```
when(repository.findById(1))  
    .thenReturn(newUser("John"));
```

Verify Interaction

```
verify(repository).findById(1);
```

Argument Matching

```
when(repository.findById(anyInt()))  
    .thenReturn(newUser("Alex"));
```